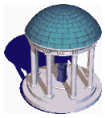


The University of North Carolina at Chapel Hill

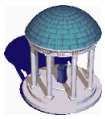
COMP 144 Programming Language Concepts

Spring 2002

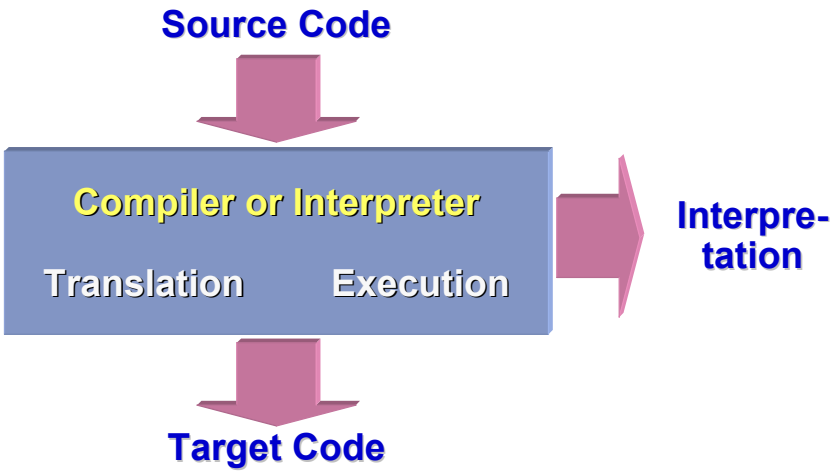
Lecture 9: Binding Time and Storage Allocation

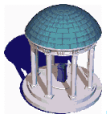
Felix Hernandez-Campos

Jan 30

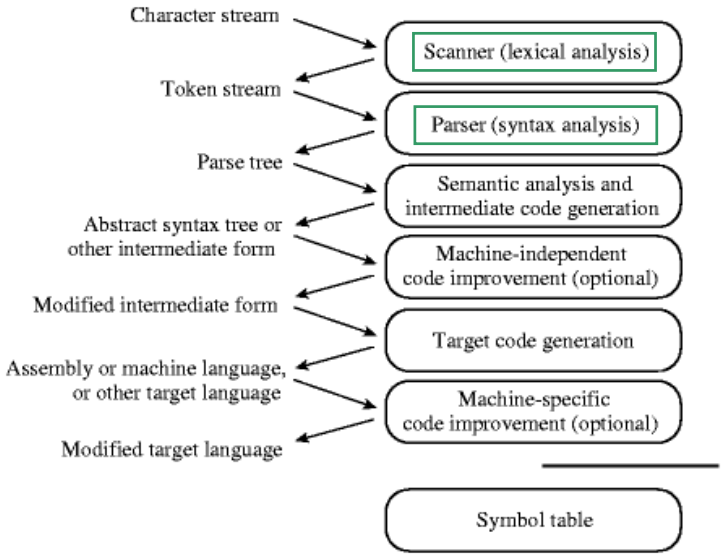


Review: Compilation/Interpretation

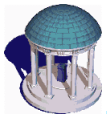




Phases of Compilation

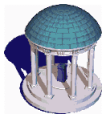


3



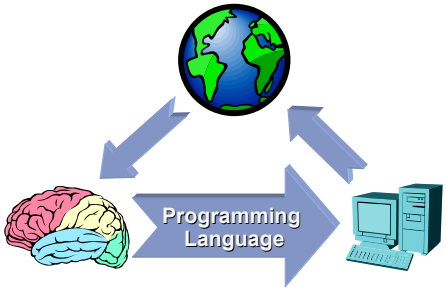
High-Level Programming Languages

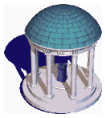
- Two main goals:
  - *Machine independence*
  - *Ease of programming*
- High-level programming language are independent of any particular instruction set
  - But compilation to assembly code requires a target instruction set
  - There is a trade-off between machine independence and efficiency
    - » *E.g.* Java vs. C



Ease of Programming

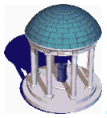
- The driving problem in programming language design
- The rest of this class
  - **Names**
  - Control Flow
  - Types
  - Subroutines
  - Object Orientation
  - Concurrency
  - Declarative Programming





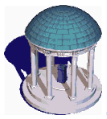
Naming

- **Naming** is the process by which the programmer associates a name with a potentially complicated program fragment
  - The goal is to hide complexity
  - Programming languages use name to designate variables, types, classes, methods, operators,…
- Naming provides *abstraction*
  - *E.g.* Mathematics is all about the formal notation (i.e. naming) that lets us explore more and more abstract concepts



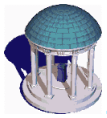
Control and Data Abstraction

- **Control abstraction** allows the programmer to hide an arbitrarily complicated code behind a simple interface
  - Subroutines
  - Classes
- **Data Abstraction** allows the programmer to hide data representation details behind abstract operations
  - ADTs
  - Classes



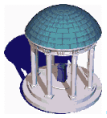
Binding Time

- A **binding** is an association between two things
  - E.g. Name of an object and the object
- **Binding time** is the time at which a binding is created
  - Language design time
  - Language implementation
  - Program writing time
  - Compile Time
  - Link Time
  - Load Time
  - Run Time



Binding Time Impact

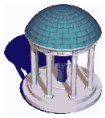
- Binding times have a **crucial** impact in programming languages
  - They are a fundamental design decision
- In general, early binding is associated with greater efficiency
  - *E.g.* C string vs. Java's StringBuffer
- In general, late binding is associated with greater flexibility
  - *E.g.* Class method vs. Subroutine
- Compiled languages tend to bind names earlier than interpreted languages



Object Lifetime

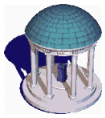
- Events in the life of an object
  - Creation
  - Destruction

  } **Object Lifetime**
- Events in the life of a binding
  - Creation
  - Destruction
    - » A binding to an object that no longer exists is called a *dangling reference*
  - Deactivation and Reactivation

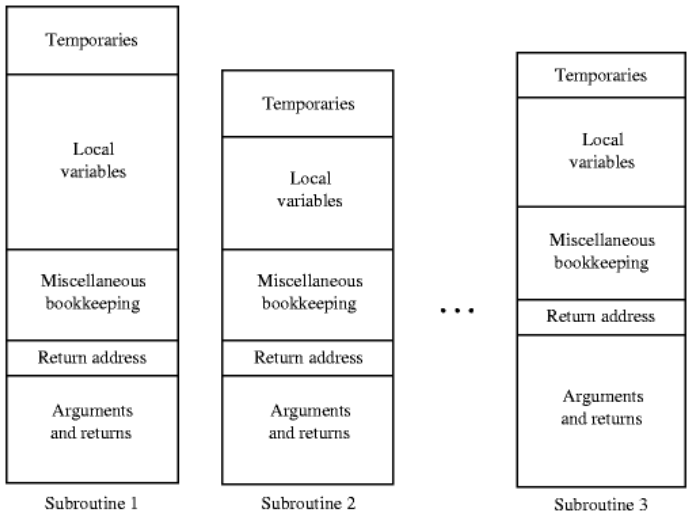


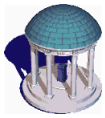
Storage Allocation Mechanisms

- In **static allocation**, objects are given an absolute address that is retained throughout the program's execution
  - *E.g.* Global variables, Non-recursive Subroutine Parameters



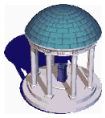
Static Allocation



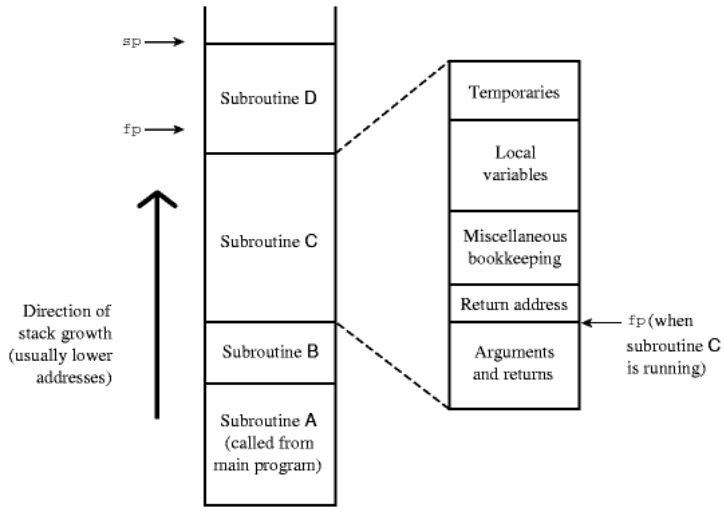


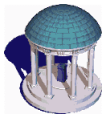
Storage Allocation Mechanisms

- In **static allocation**, (static) objects are given an absolute address that is retained throughout the program's execution
  - *E.g.* Global variables, Non-recursive Subroutine Parameters
- In **stack-based allocation**, (stack) objects are allocated in last-in, first-out data structure, a *stack*.
  - *E.g.* Recursive subroutine parameters



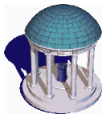
Stack-based Allocation





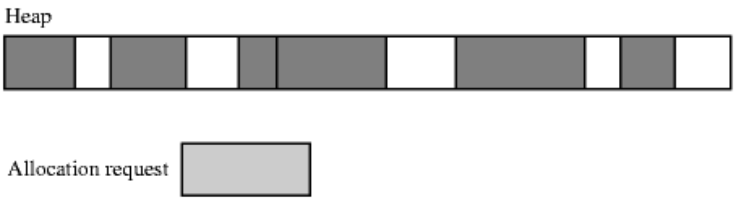
Storage Allocation Mechanisms

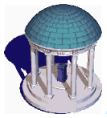
- In **static allocation**, (static) objects are given an absolute address that is retained throughout the program's execution
  - *E.g.* Global variables , Non-recursive Subroutine Parameters
- In **stack-based allocation**, (stack) objects are allocated in last-in, first-out data structure, a *stack*.
  - *E.g.* Subroutine parameters
- In **heap-based allocation**, (heap) objects may be allocated and deallocated at arbitrary times
  - *E.g.* objects created with C++ `new` and `delete`



Heap-based Allocation

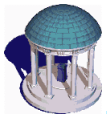
- The **heap** is a region of storage in which subblock can be allocated and deallocated
  - This not the *heap data structure*





Heap Space Management

- In general, the heap is allocated sequentially
- This creates space **fragmentation** problems
  - *Internal fragmentation*
    - » If size of object to allocated is larger than the size of the available heap
  - *External fragmentation*
    - » If size of object to allocated is not larger than the size of the available heap, but the available space in the heap is scaterred through the head in such a way that no contiguous allocation is possible
- Automatic deallocation after an object has no bindings is called *garbage collection*
  - *E.g.* Java



Reading Assignment

- Scott's chapter 3
  - Section 3.1
  - Section 3.2